Data Quality Assessment Report

E-Commerce Return Fraud Detection Dataset

Documented by: Ragavendra R

AI/ML Engineer

---

1. Summary

This report presents findings from a structured data quality assessment conducted on the E-Commerce Return Fraud Detection dataset prior to model development. The dataset contains 50,500 transaction records across 30 columns, spanning customer demographics, order details, payment information, return behaviour, and fraud labels.

Seven categories of data quality issues were identified: missing values, duplicate records, incorrect data types, invalid field values, categorical inconsistencies, mixed date formats, and statistical outliers. Each issue has been documented with its scope, root cause, and recommended remediation action. The findings form the baseline for the preprocessing pipeline that follows this assessment.

Three columns — fraud_investigation_status, refund_approved, and final_fraud_decision — were identified as post-event labels generated after fraud review completes. Their inclusion in model training would constitute data leakage and must be excluded from all feature sets.

2. Dataset Overview

Attribute	Detail
Dataset Name	ecommerce_return_fraud_dataset.csv
Total Records	50,500
Total Features	30 columns
Target Variable	return_fraud (0 = Legitimate, 1 = Fraud)
Class Distribution	Fraud: 35,487 (70.3%) | Legitimate: 15,013 (29.7%)
Data Types	8 integer, 4 float, 18 string
Memory Usage	~18.8 MB (in-memory)
Date Range (order_date)	2023 – 2026 (mixed formats present)

The target variable is heavily imbalanced — 70.3% of records are labelled as fraud. A model that always predicts fraud would achieve 70% accuracy without learning any signal. Resampling strategies (SMOTE, class weighting, or stratified splits) will be required during model training.

3. Missing Value Analysis

Four columns contain null values. All other 26 columns are fully populated. Missing rates range from 12.9% to 15.0%, which is substantial but below the threshold that would justify column removal. Imputation strategies are prescribed in Section 9.

Column	Missing Count	Missing %	Data Type	Recommended Action
customer_rating	7,575	15.00%	float64	Median imputation
customer_age	7,529	14.91%	float64	Median imputation (after removing

				invalid ages)
membership_type	6,750	13.37%	string	Mode imputation or 'Unknown' category
product_price	6,516	12.90%	string	Median imputation (after symbol cleaning)

Note: Missing values in product_price overlap with the ₹-symbol encoding issue. Symbol stripping must be performed before imputation to avoid compounding errors.

4. Duplicate Records

Duplicate detection was run at two levels: full-row identity and order_id uniqueness.

Check	Count	Risk	Action
Fully duplicate rows	500	Inflates training set; skews class ratios	Drop duplicates, keep first occurrence
Duplicate order_id values	500	Data leakage if same order appears in train and test	Investigate; remove confirmed duplicates
Customers with multiple orders	~50,000	Expected — one customer can have many orders	No action required

The 500 fully duplicate rows appear to originate from a pipeline re-ingestion event rather than legitimate re-orders, as all 30 field values are identical including timestamps. They should be removed before splitting the dataset.

5. Data Type Issues

Two numeric columns are stored as strings because of embedded symbols introduced during data collection. Both must be converted before any arithmetic operations or feature scaling.

Column	Stored As	Expected Type	Root Cause	Sample Values
product_price	string	float64	₹ currency prefix on ~7,046 rows	₹39922, ₹15278, 86585.0
discount_percent	string	float64	% suffix on ~7,057 rows	48.32%, 22.97%, 39.24

All four date columns (order_date, delivery_date, return_request_date, refund_processed_date) are stored as strings. They must be parsed to datetime objects to enable chronological validation and temporal feature engineering.

6. Invalid Field Values

Several columns contain values that are logically or physically impossible. These are distinct from statistical outliers — they represent definite data errors rather than extreme-but-valid observations.

Column	Invalid Condition	Count	Root Cause	Action
return_rate	> 1.0 (i.e., > 100%)	5,935 rows	previous_returns exceeds total_orders — calculation error	Recompute as previous_returns / total_orders; cap at 1.0

return_rate	Maximum observed: 49.0	—	Extreme calculation error (e.g., 49 returns on 1 order)	Cap after recomputation
product_price	= 99,999,999 (sentinel)	~51 rows	Placeholder value entered when price was unavailable	Replace with NaN; impute with median
product_price	= 0 (zero price)	1,026 rows	Missing price encoded as zero	Replace with NaN; impute with median
product_price	< 0 (negative price)	Rare	Data entry error	Replace with NaN
customer_age	< 18 or > 100	201 rows	Values of −10 and 200 present — clear entry errors	Replace with NaN; impute with median

7. Categorical Inconsistencies

Two columns contain variant representations of the same category — differing in case, abbreviation, or both. These will be treated as distinct classes by any encoder unless normalised first.

7.1 gender

Raw Value	Count	Correct Mapping
Male	23,655	Male
Female	22,666	Female
Male	1,428	Male
MALE	1,398	Male
M	1,353	Male

7.2 membership_type

Raw Value	Count	Correct Mapping
Gold	14,144	Gold

Silver	12,955	Silver
Platinum	12,889	Platinum
NaN	6,750	Impute or 'Unknown'
GOLD	1,279	Gold
Gld	1,249	Gold
Gold	1,234	Gold

The remaining eight categorical columns — city_tier, product_category, payment_method, return_reason, device_type, fraud_investigation_status, refund_approved, and final_fraud_decision — contain only clean, consistently cased values with no variants.

8. Date Format Issues

All four date columns store dates as strings with two co-existing formats. The majority (94%) follow the ISO standard YYYY-MM-DD, while approximately 6% use MM/DD/YYYY. Neither format is incorrect per se, but inconsistency prevents direct parsing without format inference.

Column	Dominant Format	Mixed Format	Mixed Count	Mixed %
order_date	YYYY-MM-DD	MM/DD/YYYY	3,022	6.0%
delivery_date	YYYY-MM-DD	MM/DD/YYYY	~3,000	~6.0%
return_request_date	YYYY-MM-DD	MM/DD/YYYY	~3,000	~6.0%
refund_processed_date	YYYY-MM-DD	MM/DD/YYYY	~3,000	~6.0%

After standardisation, chronological order must be validated for each record: order_date → delivery_date → return_request_date → refund_processed_date. Records where delivery_date precedes order_date, or where return_request_date precedes delivery_date, indicate temporal data

corruption and should be flagged. Future-dated entries (2026 in the current dataset) are likely pre-orders or erroneous entries and should be reviewed against the dataset reference date.

9. Outlier Analysis (IQR Method)

Outlier detection was performed using the IQR fence method: values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR are flagged. Seven of eleven numeric columns have zero outliers. The four affected columns are summarised below.

Column	Q1	Q3	IQR	Lower Fence	Upper Fence	Outlier Count	Outlier %
return_rate	0.12	0.49	0.37	−0.43	1.04	5,822	11.53 %
customer_age	31	56	25	−6.5	93.5	201	0.40%
order_amount	14,823	47,030	32,206	−33,486	95,339	68	0.13%
total_orders	51	150	99	−97.5	298.5	51	0.10%
previous_returns	12	37	25	−25.5	74.5	50	0.10%

The return_rate outliers (11.53%) are directly linked to the invalid value issue documented in Section 6 — impossible values drive the IQR fence violation. Fixing the calculation error will resolve the majority of these outliers. The remaining four columns have outlier rates below 0.5% and will be handled by capping at the IQR fence values during preprocessing.

Figure 1 — Box Plots: Outlier Distribution Across Numeric Columns

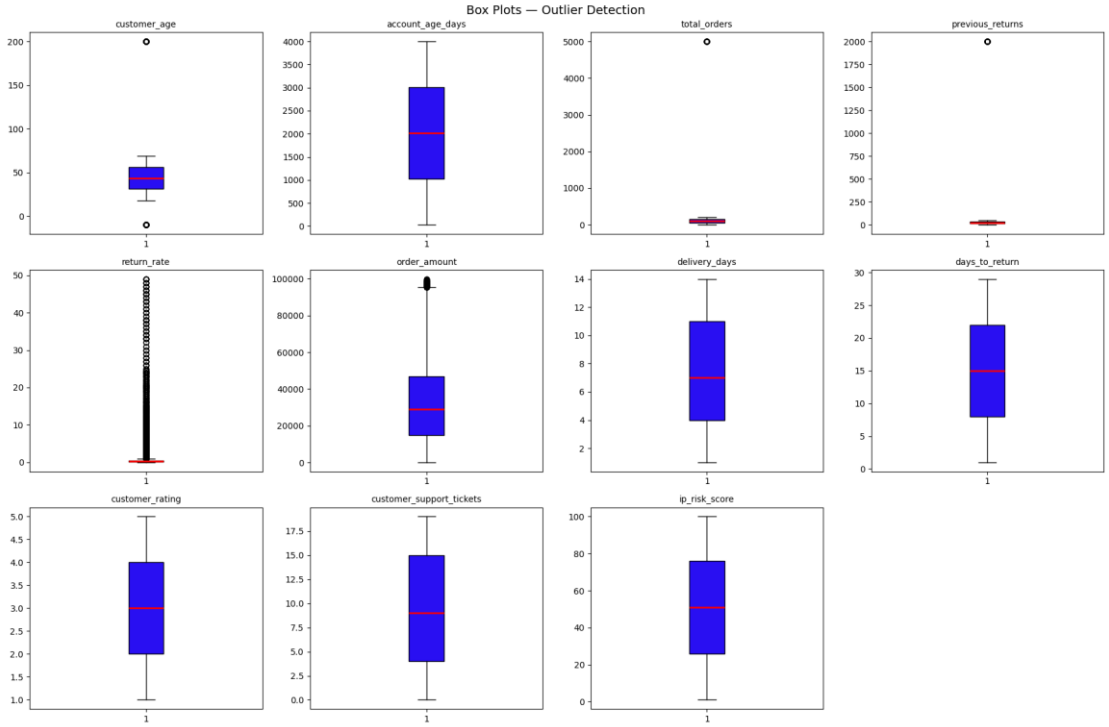

Figure 1: IQR-based outlier visualisation for all numeric columns.

10. Logical Consistency Checks

Beyond individual column quality, the following cross-column and business-logic constraints were evaluated.

Check	Result	Notes
delivery_days ≥ 1	PASS — no zero/negative values	Range: 1–14 days, no anomalies
days_to_return ≥ 1	PASS — no zero/negative values	Range: 1–29 days, no anomalies
customer_support_tickets ≥ 0	PASS — no negative values	Range: 0–19, clean
return_rate = previous_returns / total_orders	FAIL — 5,935 violations	previous_returns > total_orders in affected rows
delivery_date > order_date	To validate post date standardisation	Requires datetime conversion first

return_request_date > delivery_date	To validate post date standardisation	Requires datetime conversion first
refund_processed_date > return_request_date	To validate post date standardisation	Requires datetime conversion first

11. Data Leakage Risk

Three columns in the dataset are generated after the fraud review process completes. Including any of them as model features would cause the model to learn from the outcome rather than from pre-event signals, producing unrealistically high training accuracy that collapses entirely in production deployment.

Column	Type	When Generated	Risk Level
fraud_investigation_status	Categorical (3 values)	After investigation completes	Critical — must exclude
refund_approved	Binary (Yes/No)	After refund decision	Critical — must exclude
final_fraud_decision	Binary (Fraud/Not Fraud)	After final review	Critical — must exclude (this IS the label proxy)

12. Target Variable Distribution

Label	Value	Record Count	Percentage
Fraud	1	35,487	70.3%
Legitimate	0	15,013	29.7%
Total	—	50,500	100%

Figure 2 — Target Variable: Fraud vs Legitimate Distribution

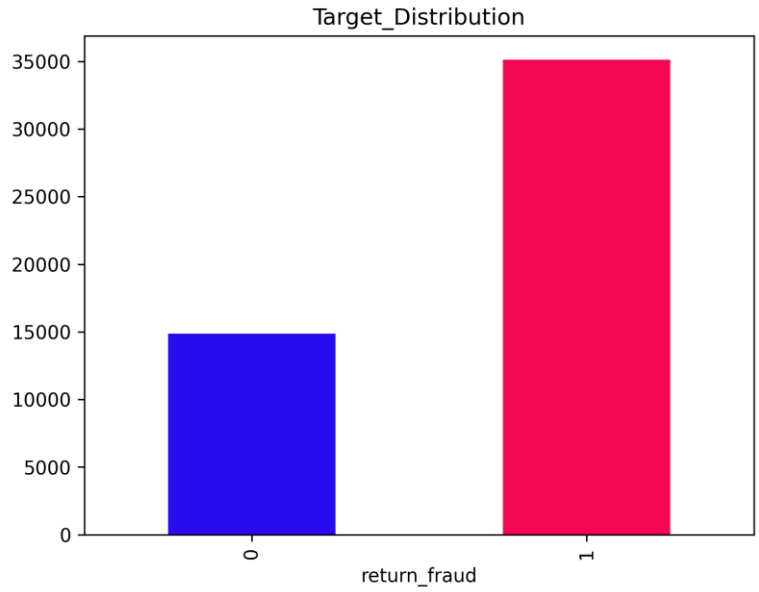

Figure 2: Class distribution of the return_fraud target variable.

The 70:30 fraud-to-legitimate imbalance is significant. Standard accuracy is not a reliable evaluation metric for this dataset. Precision, recall, F1-score (particularly for the minority class), and AUC-ROC should be used instead. Class weights or oversampling techniques such as SMOTE will be applied during model training.

13. Recommended Preprocessing Actions

Step 1 — Remove Duplicates: Drop 500 fully duplicate rows. Retain the first occurrence. Validate that no order_id appears more than once after deduplication.

Step 2 — Drop Leakage Columns: Remove fraud_investigation_status, refund_approved, and final_fraud_decision before splitting into train and test sets.

Step 3 — Fix Data Types: Strip ₹ from product_price and convert to float64. Strip % from discount_percent and convert to float64. Parse all four date columns using pd.to_datetime with format='mixed' to handle both YYYY-MM-DD and MM/DD/YYYY.

Step 4 — Fix Invalid Values: Recompute return_rate as previous_returns / total_orders and cap at 1.0. Replace product_price values of 99999999, 0, and negatives with NaN. Replace customer_age below 18 or above 100 with NaN.

Step 5 — Standardise Categoricals: Normalise gender to {Male, Female} using case-fold and abbreviation mapping. Normalise membership_type to {Gold, Silver, Platinum} using the same approach.

Step 6 — Impute Missing Values: Apply median imputation to customer_age, customer_rating, and product_price. For membership_type, impute with the mode (Gold) or add an 'Unknown' category if missingness is systematic.

Step 7 — Validate Date Logic: After parsing, confirm delivery_date > order_date, return_request_date > delivery_date, and refund_processed_date > return_request_date for every record. Flag and inspect violations.

Step 8 — Handle Outliers: Cap IQR-fence violations in total_orders, previous_returns, and order_amount. For return_rate, fixing the calculation error in Step 4 is sufficient — do not apply additional capping on top of the correction.

14. Conclusion

The dataset is workable but requires a structured eight-step cleaning pipeline before it can support reliable model training. The two most critical issues are the invalid return_rate values (affecting 11.8% of records and the most predictive feature) and the presence of three post-event leakage columns that must be excluded.

Once the preprocessing pipeline described in Section 14 is applied, the dataset will be ready for feature engineering and model development. The class imbalance (70:30) is a modelling concern rather than a data quality concern — it is an accurate reflection of the underlying fraud rate and should be addressed through appropriate evaluation metrics and resampling strategies, not by treating it as wrong data.

All cleaning steps should be implemented in a reproducible pipeline so that the same transformations can be applied consistently to any future data batches ingested into the system.